



alfamacursos.com.br



Alfama Cursos

Antônio Garcez
Fábio Garcez
Diretores Geral

Antônio Álvaro de Carvalho
Diretor Acadêmico

MATERIAL DIDÁTICO

Produção Técnica e Acadêmica

Patrícia Queiroz de Meneses
Coordenadora Geral

Patrícia Queiroz de Meneses
Coordenadora Pedagógica

Gabriella Caroline Teles Silva
Sabina Regina Conceição Santos
Revisão Textual

Rafael Rezende de Farias
Editoração

Todos os direitos reservados e protegidos pela Lei 9.610 de 19/02/98.
É proibida a reprodução total ou parcial, por quaisquer meios, sem
autorização prévia, por escrito, da
ALFAMA CURSOS.

alfamacursos.com.br



Linguagem de Programação JavaScript

APRESENTAÇÃO DO CURSO

O curso Linguagem de Programação JavaScript mostrará como deixar as suas páginas HTML muito mais dinâmicas e muito mais interessantes para os usuários que as usarem. Hoje, todos os sites e sistemas web utilizam JavaScript e durante o curso iremos conhecer um pouco do que é utilizado em todos eles.

APRESENTAÇÃO DO PROFESSOR

Tomaz Novaes Franco Pereira é graduado em Ciência da Computação pela Universidade Tiradentes (UNIT). Atualmente é desenvolvedor da Alfama Web atuando na área de desenvolvimento web por mais 3 anos, desenvolvendo sistemas em várias linguagens de programação web, como por exemplo, PHP e JavaScript. Também é um dos responsáveis pelo desenvolvimento de sistemas de sucesso, como por exemplo, o Oráculo e o Construtor de Vendas, todos esses pertencentes à empresa Alfama Web.



EMENTA:

Introdução ao JavaScript; Sintaxe, Variáveis, Constantes e Array; Valores e Conversões; Alert, Confirm e Prompt; Funções e Noções de OO; Operadores; Estruturas de controle; Objetos JavaScript (Parte I); Objetos JavaScript (Parte II); Objetos JavaScript (Parte III); Eventos JavaScript (Parte I); Eventos JavaScript (Parte II).

COMPETÊNCIAS:

- Descrever aspectos entendidos da introdução ao JavaScript.
- Aprofundar conhecimentos sobre a tecnologia JavaScript.

HABILIDADES:

- Reconhecer técnicas de programação JavaScript.

PÚBLICO-ALVO:

Estudantes de informática, técnicos de diversas áreas, empresas e repartições públicas.

SUMÁRIO

1 - Introdução ao JavaScript	2
1.1 - O que é JavaScript?	2
1.2 - JavaScript e Java	2
1.3 - Notepad++	3
1.4 - Tag <Script>	4
1.5 - Exercício Proposto	5
2 - Sintaxe, Variáveis, Constantes e Array.....	7
2.1 - Sintaxe	7
2.1.1 - Comentários.....	7
2.1.2 - Detalhes da Linguagem.....	8
2.2 - Variáveis	9
2.2.1 - O que são Variáveis?.....	9
2.2.2 - Declarando Variáveis	9
2.2.3 - Atribuindo Valor a uma Variável	10
2.2.4 - Variáveis Globais e Variáveis Locais	11
2.2.5 - Constantes	11
2.2.6 - Array	11
2.2.7 - Funções para Arrays.....	13
3 - Valores e Conversões.....	21
3.1 - Numérico (Integer, Float).....	21
3.2 - Alfanumérico (String)	21
3.3 - Lógico	21
3.4 - Null.....	22
3.5 - Undefined.....	22
3.6 - Conversões.....	22
3.7 - Parseint().....	22
3.8 - Parsefloat().....	23
4 - Alert, Confirm e Prompt	26
4.1 - Alert.....	26
4.2 - Confirm.....	26

4.3 - Prompt	27
5 - Funções e Noções de OO	30
5.1 - Funções	30
5.2 - Sintaxes	30
5.3 - A Declaração Return	35
5.4 - Noções de OO	36
5.4.1 - Orientação a Objetos	36
5.4.2 - Manipulação de Objetos	36
5.4.3 - Propriedades dos Objetos	37
5.4.4 - Métodos dos Objetos	38
6 - Operadores	40
6.1 - Introdução	40
6.2 - Operadores Aritméticos	40
6.3 - Operadores de Comparação	41
6.4 - Operadores Lógicos	42
6.5 - Operadores de Atribuição	42
6.6 - Operadores de Incremento e Decremento	43
6.7 - Exercício Proposto	44
7 - Estruturas de Controle	46
7.1 - Introdução	46
7.2 - If	46
7.3 - If...Else	47
7.4 - If...Else If...Else	49
7.5 - Switch	50
7.5.1 - Considerações Acerca da Estrutura Switch	52
7.6 - For	54
7.7 - While	56
7.8 - Do...While	58
7.9 - Break	58
7.10 - Continue	60
7.11 - Exercícios Propostos	61
8 - Objetos JavaScript (Parte I)	63
8.1 - String	63

8.1.1 - Length	63
8.1.2 - Charat (Posição).....	63
8.1.3 - Indexof("String")	63
8.1.4 - Substring (Index1, Index2).....	64
8.1.5 - Touppercase()	64
8.1.6 - Tolowercase()	64
8.1.7 - Escape().....	65
8.1.8 - Replace()	65
8.2 - Math.....	66
8.2.1 - Math.Abs(Número).....	66
8.2.2 - Math.ceil(Número).....	66
8.2.3 - Math.floor(Número).....	66
8.2.4 - Math.round(Número)	66
8.2.5 - Math.Pow(Base, Expoente).....	66
8.2.6 - Math.max(Número1, Número2)	67
8.2.7 - Math.min(Número1, Número2).....	67
8.2.8 - Math.sqrt(Número)	67
8.2.9 - Math.sin(Número).....	67
8.2.10 - Math.asin(Número)	67
8.2.11 - Math.cos(Número)	68
8.2.12 - Math.acos(Número)	68
8.2.13 - Math.tan(Número)	68
8.2.14 - Math.atan(Número).....	68
8.2.15 - Math.pi	68
8.2.16 - Math.log(Número)	68
8.2.17 - Math.e.....	68
8.2.18 - Math.random()	68
8.3 - Exercícios Propostos	69
9 - Objetos Javascript (Parte II).....	71
9.1 - Date	71
9.1.1 - Métodos do Objeto Date().....	72
9.2 - Document.....	74
9.2.1 - Propriedades do Objeto Document	75

9.2.2 - Coleções do Objeto Document	75
9.2.3 - Métodos do Objeto Document.....	76
9.3 - Exercício Proposto	77
10 - Objetos JavaScript (Parte III)	79
10.1 - Location	79
10.1.1 - Propriedades do Objeto Location	79
10.1.2 - Métodos do Objeto Location	80
10.2 - Window.....	81
10.2.1 - Propriedades do Objeto Window	81
10.2.2 - Coleções do Objeto Window	83
10.2.3 - Métodos do Objeto Window.....	83
11 - Eventos JavaScript (Parte I)	87
11.1 - Introdução	87
11.2 - Onclick.....	87
11.3 - Onkeydown, Onkeypress e Onkeyup	87
11.4 - Onmouseover e Onmouseout	88
12 - Eventos JavaScript (Parte II).....	91
12.1 - Onsubmit.....	91
12.2 - Onfocus, Onblur e Onchange.....	93
12.3 - Onload.....	94
12.4 - Onfocus.....	95
12.5 - Exercícios Propostos	96
Referências Bibliográficas	97

Capítulo 1

Introdução ao JavaScript

1 - INTRODUÇÃO AO JAVASCRIPT

1.1 - O QUE É JAVASCRIPT?

Desenvolvida pela Netscape, a linguagem JavaScript, a princípio chamado de LiveScript, foi criada para trabalhar com aplicações interativas nas páginas HTML. Esta linguagem teve sua primeira versão desenvolvida para o browser Netscape Navigator 2.0 e em seguida, atribuído também ao Internet Explorer 3.0.

JavaScript é uma linguagem client-side, ou seja, ela trabalha (roda, funciona) ao lado do cliente. Isso significa dizer que ela é executada no browser – e não no servidor – ao contrário de algumas linguagens, como o PHP, ASP, JSP, .NET que são executadas no servidor (server-side).

O cliente tem total controle do suporte a essa linguagem no browser utilizado, podendo desabilitar o funcionamento desse recurso, caso queira.

Usando o JavaScript, é possível modificar dinamicamente os estilos dos elementos da página em HTML.

Os principais browsers do mercado – Firefox, Mozilla, Opera, Netscape, Safari, Internet Explorer, etc. – suportam a linguagem JavaScript. Ela é utilizada para avaliar formulários, ler e inserir conteúdo em uma página, criar *cookies*, menus dinâmicos e outros diversos recursos.

1.2 - JAVASCRIPT E JAVA

Antes de aprender as diferenças entre essas duas linguagens é bom que você conheça a diferença entre linguagem compilada e linguagem interpretada. Linguagens compiladas são aquelas como Delphi, Visual Basic, C++ (dentre outras). O Java pertence a esse grupo de linguagens que, geralmente, produzem uma executável, ou seja, um arquivo com a extensão .exe. Já o JavaScript é uma linguagem puramente interpretada.

Linguagens interpretadas (também chamadas linguagens de script) são linguagens de programação executadas em um interpretador. Os scripts, que são as instruções formais escritas com as linguagens interpretadas, são diferentes de programas de computador: enquanto os programas de computador são convertidos em formatos executáveis binários para

serem usados, os scripts mantêm seu formato e são interpretados comando a comando cada vez que são executados.

As linguagens interpretadas foram criadas para simplificar o ciclo tradicional de programação "edição - compilação - ligação - execução". Entretanto, nada impede que uma linguagem interpretada seja compilada para o código de máquina, ou que uma linguagem de programação tradicional seja interpretada sem compilação.

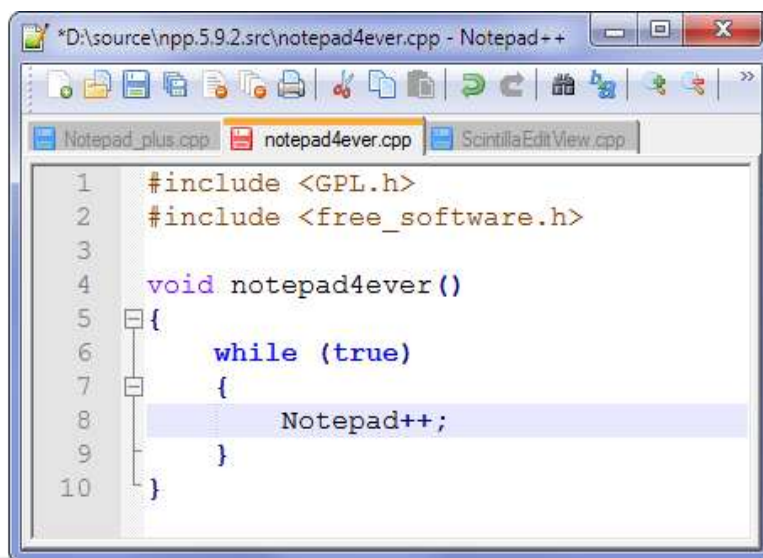
Entre as linguagens interpretadas mais populares estão Perl, Python, PHP, JavaScript e Tcl.

1.3 - NOTEPAD++

Notepad++ é um editor de texto e de código-fonte de código aberto sob a licença GPL. Suporta várias linguagens de programação rodando sob o sistema Microsoft Windows.

As linguagens de programação suportadas pelo Notepad++ são: C, C++, Java, C#, XML, HTML, PHP, JavaScript, Makefile, ASCII art, Doxygen, ASP, VB/VBScript, Unix Shell Script, BAT, SQL, Objective-C, CSS, Pascal, Perl, Python, Lua, Tcl, Assembly, Ruby, Lisp, Scheme, Smalltalk, PostScript e VHDL.

Ele suporta autocomplemento, busca e substituição de expressões regulares, divisão de tela, zoom, favoritos, etc. Tem suporte para macros e plugins.



1.4 - TAG <SCRIPT>

Para inserir o JavaScript em uma página HTML usa-se a tag <script> e o atributo type com o valor: text/javascript. Assim:

```
<script type="text/javascript">
```

```
    // Comandos
```

```
</script>
```

Ao iniciar a tag <script> informe o tipo de script a ser utilizado: type="text/javascript"; assim o browser entenderá que há comandos JavaScript para serem executados e os executará. Finalize a execução dos códigos com a seguinte tag: </script>.

A tag <script> pode ser adicionada na página HTML várias vezes e em vários locais da página, mas normalmente é utilizada dentro da tag <head>.

Outra maneira de inserir o JavaScript é por meio de documentos externos, também utilizando a tag <script>. O arquivo externo é chamado para a página desta forma:

```
<html>
```

```
    <head>
```

```
        <title>Alfama Cursos</title>
```

```
        <script type="text/javascript" src="arquivo.js"></script>
```

```
    </head>
```

```
    <body>
```

```
    </body>
```

```
</html>
```

O atributo src indica a localização do arquivo externo para que o browser possa identificá-lo e executá-lo. Obs.: Em um arquivo externo não se utiliza a tag <script>, somente os códigos. Vantagens de trabalhar usando arquivo externo (.js):

- **Facilidade na manutenção:** uma vez que o script está localizado em apenas um arquivo, facilita a edição ou correção dos códigos.

- **Carregamento mais rápido da página:** o arquivo externo é armazenado no cache do navegador; assim, evita-se carregá-lo toda vez que a página for chamada.
- **Semântico:** o arquivo externo separa a camada de comportamento (JavaScript) da camada de conteúdo (HTML).

Obs.: As tag's <html>, <head>, <title> e <body> são as tag's básicas de uma página HTML.

Uma página HTML com a estrutura básica é escrita dessa forma:

```
<html>

    <head>

        <title>Alfama Cursos</title>

    </head>

    <body>

    </body>

</html>
```

1.5 - EXERCÍCIO PROPOSTO

- 1) Instale o Notepad++, crie uma página HTML (adicione as tag's da estrutura básica HTML) e adicione a essa página a tag <script> dentro da tag <head> e depois adicione mais uma tag <script> chamando um arquivo externo .js.

Capítulo 2

Sintaxe, Variáveis, Constantes e Array

2 - SINTAXE, VARIÁVEIS, CONSTANTES E ARRAY

2.1 - SINTAXE

2.1.1 - COMENTÁRIOS

Os comentários na linguagem JavaScript têm função similar aos das demais linguagens, ou seja: explicar o que determinado comando faz, lembretes, dentre outros.

Os comentários em uma linha são aqueles que você usa apenas quando precisa de um comentário rápido e simples. Como o próprio nome já diz, eles ocupam apenas uma linha (embora possam ser tão longos quanto você queira) e, normalmente, são curtos e diretos. Veja:

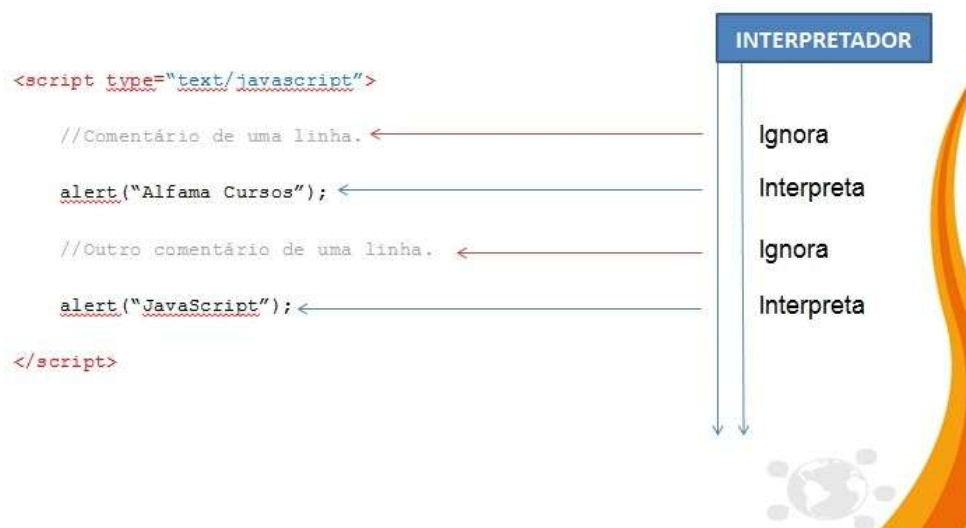
```
<script type="text/javascript">

    //Comentário de uma linha.

    //Outro comentário, de uma linha.

</script>
```

O interpretador do JavaScript, ao ver as duas barras (//), ignorará tudo o que está depois delas até o final da linha.



Comentários de bloco são empregados quando se quer inserir um comentário mais detalhado sobre uma função, ou mesmo sobre o arquivo JavaScript como um todo.

Para abrir um comentário em bloco usa-se o `/*` e para fechar o comentário usa-se `*/`.

```
<script type="text/javascript">
```

```
    /*
```

```
    Comentário de bloco...
```

```
    Alfama Cursos
```

```
    Cursos de JavaScript
```

```
    */
```

```
</script>
```

2.1.2 - DETALHES DA LINGUAGEM

JavaScript é "case-sensitive"; isso significa que o interpretador diferencia as letras minúsculas e maiúsculas. Uma simples letra maiúscula, ou minúscula, faz toda a diferença.

Em JavaScript a utilização do `;` (ponto e vírgula) é quase opcional. Quase porque se você informar duas declarações de código na mesma linha precisará do `;` (ponto e vírgula).

Veja o exemplo abaixo:

```
<html>
```

```
    <head>
```

```
        <title>Alfama Cursos - JavaScript</title>
```

```
        <script type="text/javascript">
```

```
            alert("Não vou usar (;)")
```

```
            alert("Vou usar (;)");
```

```
        </script>
```

```
</head>
```

```
<body>
```

```
</body>
```

```
</html>
```

É recomendado utilizar o ponto e vírgula (;) no final das declarações, pois 90% das linguagens de programação o utilizam.

2.2 - VARIÁVEIS

2.2.1 - O QUE SÃO VARIÁVEIS?

Variáveis são de grande importância. Com esta metáfora você entenderá mais facilmente: "Imagine uma estante em que você guarda diversos livros: ao ter um novo livro, você o coloca na estante; quando quiser lê-lo, basta pegá-lo na estante".

Com o computador é a mesma coisa: a memória RAM é a estante e as variáveis e constantes são os livros; você cria (obtem) e as joga na memória. Claro que o computador faz isso sozinho, você só precisa dizer o nome e o tipo da variável.

Variáveis são como um depósito que você utiliza para guardar informações, e o seu valor (conteúdo) pode ser alterado durante a execução do script.

2.2.2 - DECLARANDO VARIÁVEIS

Veja um exemplo:

```
<script type="text/javascript">
```

```
var nomedavariavel = "Valor da variável";
```

```
//podemos declarar assim também:
```

```
nomedavariavel = "Valor da variável";
```

</script>

Você pode criar uma variável com ou sem a declaração var, mas é aconselhável utilizá-la.

Regras necessárias para criação de variáveis:

- JavaScript é "case-sensitive", como no PHP, ou seja, uma variável com o nome Alfama é diferente de alfama.
- O nome de uma variável deve iniciar com uma letra ou com um sublinhado (_).

Exemplos de nomes de variáveis válidas:

<script type="text/javascript">

var casa120 = "valor";

var variavel = "valor";

var _variavel = "valor";

</script>

Exemplos de nomes de variáveis inválidas:

<script type="text/javascript">

var 20teconhecer = "valor";

var 100vergonha = "valor";

var 13zentos = "valor";

</script>

Obs.: O JavaScript tem tipagem dinâmica, não sendo preciso escolher o tipo de dado da variável.

2.2.3 - ATRIBUINDO VALOR A UMA VARIÁVEL

O valor “Alfama Cursos” será atribuído à variável treinamento. Veja:

```
<script type="text/javascript">
```

```
    var treinamento = " Alfama Cursos ";
```

```
</script>
```

2.2.4 - VARIÁVEIS GLOBAIS E VARIÁVEIS LOCAIS

Os nomes global e local são basicamente autoexplicativos, mas você deve compreendê-los bem porque são de grande importância.

Quando uma variável é criada dentro de uma função ela se torna local, ou seja, acessível somente dentro daquela função. Pode até serem criadas outras variáveis com o mesmo nome e com valores diferentes, porém elas terão validade somente dentro da função em que foram declaradas. Já as variáveis globais são declaradas fora das funções e ficam acessíveis a toda e qualquer função na página em que as variáveis estejam.

2.2.5 - CONSTANTES

São declaradas com a palavra reservada **const**. Não podem sofrer alteração de seu conteúdo. Exemplo:

```
<script type="text/javascript">
```

```
    const curso = "JavaScript";
```

```
    const ano = "2012";
```

```
</script>
```

2.2.6 - ARRAY

Um Array possibilita armazenar diversos valores em uma única variável. Dessa forma seria uma variável com vários valores, armazenados e acessados através de uma ordem, de um índice. Imagine uma variável com o nome diasDaSemana em que será preciso armazenar sete valores (de segunda a domingo); a solução está em utilizar o Array para definir sete valores em uma única variável. Define-se um Array através da palavra-chave new. Sintaxe:

```
<script type="text/javascript">
```

```
var diasDaSemana = new Array();
```

```
var diasDaSemana = new Array(7);
```

```
</script>
```

Os dois exemplos acima estão corretos. A diferença entre eles é que o segundo informa o tamanho que o Array deve ter (7), neste caso. Construindo o Array dias da semana:

```
<html>
```

```
<head>
```

```
<title> Alfama Cursos - JavaScript </title>
```

```
<script type="text/javascript">
```

```
var diasDaSemana = new Array();
```

```
diasDaSemana[0] = "Domingo";
```

```
diasDaSemana[1] = "Segunda";
```

```
diasDaSemana[2] = "Terça";
```

```
diasDaSemana[3] = "Quarta";
```

```
diasDaSemana[4] = "Quinta";
```

```
diasDaSemana[5] = "Sexta";
```

```
diasDaSemana[6] = "Sábado";
```

```
alert(diasDaSemana[6]);
```

```
</script>
```

```
</head>
```

```
<body>
```

```
</body>
```

```
</html>
```

Para exibir o valor de uma chave deve-se fazer o seguinte: diasDaSemana[1] //. O resultado será segunda-feira. Neste exemplo foi construído um Array com 7 sete chaves; cada uma tem seu valor armazenado (neste caso, os dias da semana). Depois foi dado um alert() com o valor do Array selecionado. Outro exemplo:

```
<html>

  <head>

    <title> Alfama Cursos - JavaScript </title>

    <script type="text/javascript">

      var diasDaSemana = new Array("Domingo", "Segunda", "Terça",
      "Quarta", "Quinta", "Sexta", "Sábado");

      alert(diasDaSemana[6]);

    </script>

  </head>

  <body>

  </body>

</html>
```

2.2.7 - FUNÇÕES PARA ARRAYS

Algumas das principais funções para Arrays:

- **join():** junta sequencialmente os elementos de um Array usando o caractere (ou caracteres) passado por parâmetro. Se ele não é indicado, usa-se a vírgula como default.
- **shift():** retorna o primeiro valor de um Array e remove os demais.
- **sort():** ordena os elementos de um Array.
- **reverse():** inverte a ordem dos elementos. O primeiro se torna o último e o último se torna o primeiro.

- **slice():** retorna os elementos selecionados de um Array.
- **length():** indica o número de elementos em um Array.
- **push (valor1, valor2, ...):** adiciona um ou mais elementos (valor1, valor2, etc.) ao final de um Array, retornando o seu novo tamanho.
- **pop():** retorna o último elemento de um Array e remove os demais.

Exemplo: join()

```
<html>
  <head>

    <title> Alfama Cursos - JavaScript </title>

    <script type="text/javascript">

      var arr = new Array();

      arr[0] = "Alfama";

      arr[1] = "Cursos";

      arr[2] = " PHP ";

      arr[3] = "JavaScript";

      document.write(arr.join("<br />"));

      document.write("<br /><br />");

      document.write(arr.join(" <> "));

    </script>

  </head>

  <body>
```



```
</body>
```

```
</html>
```

Veja que todo o conteúdo do Array foi colocado em forma de string, que foram impressos no documento. No primeiro join cada item do Array foi separado por um espaço
. No segundo join, foi separado pelos caracteres <>.

Exemplo: shift()

```
<html>
```

```
<head>
```

```
<title> Alfama Cursos - JavaScript </title>
```

```
<script type="text/javascript">
```

```
var arr = new Array(3);
```

```
arr[0] = "Alfama";
```

```
arr[1] = "Cursos";
```

```
arr[2] = " PHP ";
```

```
arr[3] = "JavaScript";
```

```
document.write(arr.shift());
```

```
</script>
```

```
</head>
```

```
<body>
```

```
</body>
```

```
</html>
```

No exemplo em estudo, shift() retornará apenas o primeiro valor do Array.

Exemplo: reverse()

```
<html>
  <head>

    <title> Alfama Cursos - JavaScript </title>

  </head>

  <body>

    <script type="text/javascript">

      var arr = new Array(3);

      arr[0] = "Alfama";

      arr[1] = "Cursos";

      arr[2] = " PHP ";

      arr[3] = "JavaScript";

      document.write(arr.reverse());

    </script>

  </body>

</html>
```

Neste exemplo, ele mostrará todos os itens do Array, mas não ordenados, mostrará do último ao primeiro.

Exemplo: slice()

Extraí uma parte de um Array e retorna um novo com essa parte. O parâmetro ini é o índice base zero a partir do qual se inicia a extração. Os elementos extraídos não incluem o índice base zero dado por fim. Se ele não é indicado, a operação se dá até o final da sequência.

A função não altera o objeto original. Os valores são copiados para o novo Array.

Exemplo: Escreva: "Cursos, PHP":

```
<html>

  <head>

    <title> Alfama Cursos - JavaScript </title>

  </head>

  <body>

    <script type="text/javascript">

      var arr = new Array(3);

      arr[0] = "Alfama";

      arr[1] = "Cursos";

      arr[2] = " PHP ";

      arr[3] = "JavaScript";

      document.write(arr.slice(1,3));

    </script>

  </body>

</html>
```

Exemplo: length()

```
<html>

  <head>

    <title> Alfama Cursos - JavaScript </title>

  </head>

  <body>

    <script type="text/javascript">

      var arr = new Array();
```

```

arr[0] = "Alfama";

arr[1] = "Cursos";

arr[2] = " PHP ";

arr[3] = "JavaScript";

alert(arr.length); //Exibe o tamanho atual do array, neste caso 4

arr.length = 3; //Diminui o seu tamanho

alert(arr.length); //Exibe o tamanho atual do array, neste caso 3

document.write(arr); //Ao diminuirmos o tamanho do Array, o elemento
"JavaScript" foi retirado.

</script>

```

```

</body>

```

```

</html>

```

Exemplo: push()

```

<html>

```

```

  <head>

```

```

    <title> Alfama Cursos - JavaScript </title>

```

```

  </head>

```

```

  <body>

```

```

    <script type="text/javascript">

```

```

      var arr = new Array(3);

```

```

      arr[0] = "Alfama";

```

```

      arr[1] = "Cursos";

```

```

      document.write(arr); //Elementos iniciais do array
    </script>
  </body>
</html>

```

```
document.write("<br /><br />");

document.write("<b>Array com novos elementos:</b><br /><br />");

arr.push("PHP","JavaScript","HTML");

document.write(arr);

</script>

</body>

</html>
```

RECORDANDO

Usa-se os comentários de linha (//Comentário) ou de bloco (/* Comentário */) para explicar o que o código escrito realiza. Sempre no final de cada linha de comando coloca-se o “ ; ”. Uma variável pode ser ou não declarada utilizando a palavra reservada “ **var** ”. Array é uma variável que possibilita armazenar vários valores, cada valor é adicionado em um índice do Array.

Capítulo 3

Valores e Conversões

3 - VALORES E CONVERSÕES

Os valores das variáveis JavaScript podem ser do tipo numérico (integer, float), alfanumérico (string) e lógico (boolean). Além desses valores as variáveis também podem receber o valor NULL ou ser UNDEFINED.

3.1 - NÚMÉRICO (INTERGER, FLOAT)

São os valores numéricos, podendo ser números negativos ou de ponto flutuante.

```
var numero = 10;
```

```
var numero = -10;
```

```
var numero = 8.2;
```

3.2 - ALFANUMÉRICO (STRING)

São os valores alfanuméricos, strings, sempre representados entre “ ” (aspas duplas) ou ‘ ’ (aspas simples).

```
var nome = “Alfama”;
```

```
var nome = ‘Alfama’;
```

```
var nome = “Alfama 2012”;
```

3.3 - LÓGICO

São os valores booleanos, podem ser apenas verdadeiros (true) ou falsos (false).

```
var logica = true;
```

```
var logica = false;
```

3.4 - NULL

Representa o valor nulo quando a variável não armaneza valor nenhum.

```
var nome = null;
```

3.5 - UNDEFINED

É quando o valor da variável não é definido, quando a variável é apenas declarada.

```
var nome;
```

3.6 - CONVERSÕES

Como o JavaScript é de tipagem fraca, algumas conversões são feitas de forma automática.

“10” + 8 = “108” – O número “10” está entre aspas, então o JavaScript entende que é uma string e não realiza a soma dos valores e sim a concatenação, para isso ele converte o valor integer 8 que está sem aspas simples para uma string “8”. Concatenando a string “10” com a estring “8” o resultado será a string “108”.

“18” – 8 = 10 – O número “18” está entre aspas, então o JavaScript entende que é uma string, porém como está sendo feita uma subtração, o valor é convertido de string para integer e a subtração é realizada.

3.7 - PARSEINT()

Com a função parseInt, o usuário poderá converter valores de string em valores numéricos inteiros. Caso a função encontre um caractere diferente de um sinal (+ ou -), números (0 à 9), ponto decimal ou expoente, será retornado o valor até aquele ponto e ignorado o restante. Caso o primeiro caractere não possa ser convertido para um número, a função parseInt retornará o valor NaN (Not-A-Number [Não é um número]).

No exemplo a seguir é convertido um valor string em seu valor numérico equivalente:

```
<html>
```

```
<head>
```



```

<title> Alfama Cursos - JavaScript </title>

</head>

<body>

    <script type="text/javascript">

        var valor = parseInt("123");

        alert(valor);

        var valor = parseInt("1.23");

        alert(valor);

    </script>

</body>

</html>

```

3.8 - PARSEFLOAT()

Com a função `parseFloat`, é feita a conversão de uma string, retornando um valor de ponto flutuante. Com ela é convertida uma string em um valor numérico equivalente. Se a função encontrar um caractere diferente de um sinal (+ ou -), números (0 a 9), ponto decimal ou expoente, será retornado o valor até aquele ponto e ignorado o restante. Caso o primeiro caractere não possa ser convertido para um número, o `parseFloat` retornará o valor NaN (Not-A-Number [Não é um número]).

Veja a seguir um exemplo da utilização da função `parseFloat`.

```

<html>

    <head>

        <title> Alfama Cursos - JavaScript </title>

    </head>

    <body>

```

```

<script type="text/javascript">

    var valor = parseFloat("123");

    alert(valor);

    var valor = parseFloat ("1.23");

    alert(valor);

</script>

</body>

</html>

```

RECORDANDO

As variáveis JavaScript podem ser do tipo numérico, alfanumérico e lógico, podem também ter seu valor nulo (NULL) e indefinido (UNDEFINED). Algumas conversões no JavaScript são feitas de forma automática. As funções parseInt e parseFloat convertem as strings em números.

Capítulo 4

Alert, Confirm e Prompt

4 - ALERT, CONFIRM E PROMPT

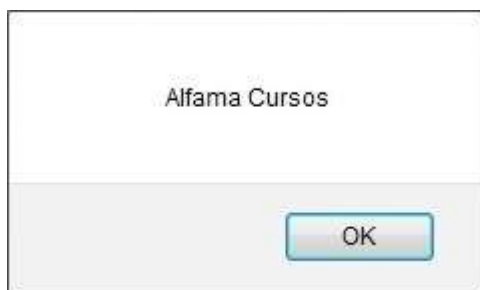
Permitem alertar o usuário sobre algum acontecimento, pedir-lhe uma confirmação para que determinada ação possa ser executada e pedir-lhe que informe algum dado.

4.1 - ALERT

Sintaxe:

```
alert("Alfama Cursos");
```

Em execução:



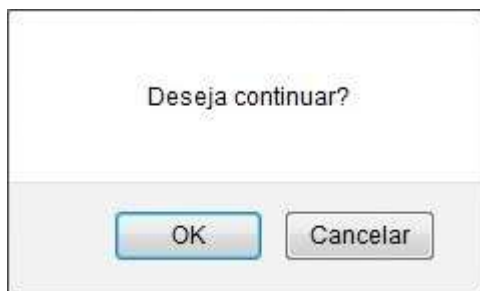
Como sugere a própria sintaxe "alert", o objetivo é alertar o usuário sobre algo. Ele possui um botão "OK" para prosseguir com a navegação.

4.2 - CONFIRM

Sintaxe:

```
confirm("Deseja continuar?");
```

Em execução:



Com o Confirm pode-se ter certeza se o usuário deseja executar determinada ação, visto que esse box exibe dois botões: OK e Cancelar. Ao clicar em OK o box retorna true (verdadeiro) e ao clicar em Cancelar o box retorna false (falso).

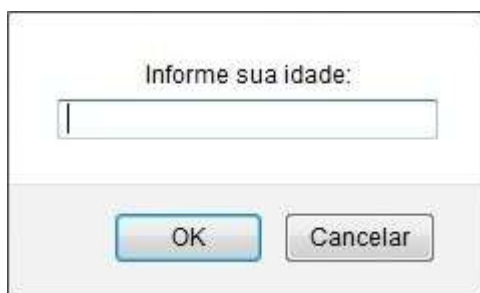
4.3 - PROMPT

Sintaxe:

```
prompt("Informe sua idade:"); //Sem sugestão de resposta
```

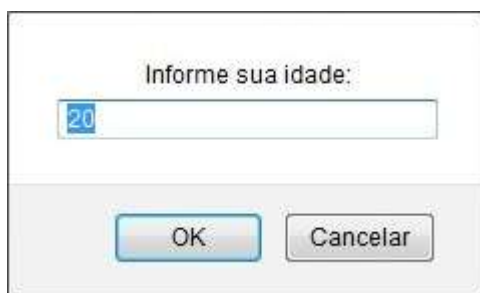
```
prompt("Informe sua idade:", "20"); //Com sugestão de resposta
```

Em execução: Sem sugestão



The image shows a standard JavaScript prompt dialog box. At the top, it says "Informe sua idade:". Below this is a single-line text input field that is currently empty. At the bottom of the dialog, there are two buttons: "OK" on the left and "Cancelar" on the right.

Em execução: Com sugestão



The image shows a JavaScript prompt dialog box similar to the one above, but with a default value. The text input field now contains the number "20". The "Informe sua idade:" label and the "OK" and "Cancelar" buttons are also present.

É similar ao Confirm, porém o valor retornado é definido pelo próprio usuário ou pela resposta padrão.

RECORDANDO

O Alert é uma janela de alerta ao usuário, o Confirm também é uma janela de alerta com a opção de o usuário responder uma confirmação, já o Prompt permite o usuário entrar com uma informação, tendo a opção de ter uma resposta padrão.

Capítulo 5

Funções e Noções de OO

5 - FUNÇÕES E NOÇÕES DE OO

5.1 - FUNÇÕES

Funções são partes muito importantes de uma linguagem de programação. Todas as linguagens de programação têm os seus conjuntos de funções. Mas, o que é uma função? Pode-se definir Funções como trechos de código que você pode executar, fazendo a chamada por meio de um link, um botão ou até mesmo dentro de uma outra função. O JavaScript tem um bom número de funções built-in, ou seja, funções que já fazem parte da linguagem. Mas a grande parte das funções que forem utilizadas será escrita por você. As funções built-in serão explicadas em outro momento deste curso. Você pode chamar uma função de qualquer lugar da página, ou até mesmo de outras páginas, contanto que o script esteja em um arquivo externo (.js).

5.2 - SINTAXES

As funções podem conter parâmetros, ou não; quando eles existem são informados à função quando ela é chamada. Acompanhe duas sintaxes para a criação de funções e os exemplos com, e sem, parâmetros.

- **Sem parâmetros**

```
function nome_da_funcao() {  
  
    //Comandos da função  
  
}
```

- **Com parâmetros**

```
function nome_da_funcao(variavel1,variavel2,...) {  
  
    //Comandos da função  
  
}
```

Exemplo:

```
<html>
```



```
<head>

    <title> Alfama Cursos - JavaScript </title>

    <script type="text/javascript">

        function BemVindo(){

            alert("Seja bem vindo!");

        }

    </script>

</head>

<body>

    <p onClick="BemVindo()">Clique aqui ...</p>

</body>

</html>
```

Foi criada uma função simples que, ao ser chamada, exibe um alerta. Ela é invocada através do evento onclick inserido no parágrafo <p>.

Exemplo:

```
<html>

    <head>

        <title> Alfama Cursos - JavaScript </title>

    </head>

    <body>

        <script type="text/javascript">

            function Alerta() {

                var pergunta = confirm("Deseja fazer uma busca?");
```

```

        if (pergunta) {

            window.location.href = "http://www.google.com.br";

        }

    }

</script>

<a href="javascript:Alerta();">Google Search</a>

</body>

</html>

```

Nesse exemplo, a função é invocada a partir de um link, que quando clicado, pergunta se deseja fazer uma pesquisa / busca; se sim, clicar em OK e ele abrirá o Google.

Exemplo:

```

<html>

<head>

    <title> Alfama Cursos - JavaScript </title>

</head>

<body>

    <script type="text/javascript">

        function LogarAluno(strnome) {

            alert("Olá" + strnome + ", Seja bem vindo!");

            window.location.href = "http://www.alfamaweb.com.br";

        }

    </script>

    <form name="formaluno">

```

```

<strong>Digite seu nome:</strong>

<input type="text" name="txtnome" size="20">

<br />

<br />

<input      type="submit"      value="Entrar"      name="botao"
onClick="LogarAluno(document.formaluno.txtnome.value);">

</form>

</body>

</html>

```

Para enviar o nome digitado à função faça o seguinte:

LogarAluno(document.nomeForm.nomeCampo.value) document é igual ao documento, ou seja, a página em si; nomeForm é igual ao nome do form, concedido através do atributo name da tag form; nomeCampo é o nome do campo no qual o usuário informa seu nome; já o nomeCampo.value é o valor contido no campo, no caso, o nome informado. Ao clicar no botão "entrar" é invocada a função LogarAluno, passando a ela o nome digitado no input: txtnome.

Exemplo:

```

<html>

<head>

<meta http-equiv="Content-Language" content="en-us">

<title> Alfama Cursos - JavaScript </title>

</head>

<body>

<script type="text/javascript">

function Idade(Anos) {

```

```

        if (Anos >= 18) {

            alert("Maior de Idade");

        } else {

            alert ("Menor de Idade")

        }

    }

</script>

<!-- formulário: formverifica -->

<form name="formverifica">

    Qual sua idade ?

    <br />

    <!--Edit, onde você digitará sua idade -->

    <input type="text" name="txtidade" size="20">

    <br />

    <br />

    <!-- Ao clicar no botão, ele chama a função -->

    <input      type="submit"      value="Verificar"      name="botao"
onClick="Idade(document.formverifica.txtidade.value)">

</form>

</body>

</html>

```

Mesma linha de raciocínio do exemplo anterior.

5.3 - A DECLARAÇÃO RETURN

Quando se quer que uma função retorne algum valor, utiliza-se a declaração return, que tem o papel de especificar o valor retornado pela função. Exemplo:

```
<html>

  <head>

    <title> Alfama Cursos - JavaScript </title>

  </head>

  <body>

    <script type="text/javascript">

      function soma(valor1,valor2) {

        total = valor1 + valor2;

        return total;

      }

    </script>

    <a href="javascript: alert(soma(25,25))">Resultado</a>

  </body>

</html>
```

No exemplo, a função soma() é responsável por obter e somar os valores recebidos como parâmetros: "valor1" e "valor2". Ela retornará um valor. Usa-se um alert para exibir o resultado da soma dos valores enviados a essa função. Considerações finais sobre funções:

- A palavra function sempre deve ser escrita com letras minúsculas.
- O nome da função pode conter letras minúsculas e/ou maiúsculas.
- As chaves { } indicam o início e o término da função.

- Mesmo que uma função não contenha parâmetros, os parênteses () devem ser incluídos após o nome da função.

5.4 - NOÇÕES DE OO

5.4.1 - ORIENTAÇÃO A OBJETOS

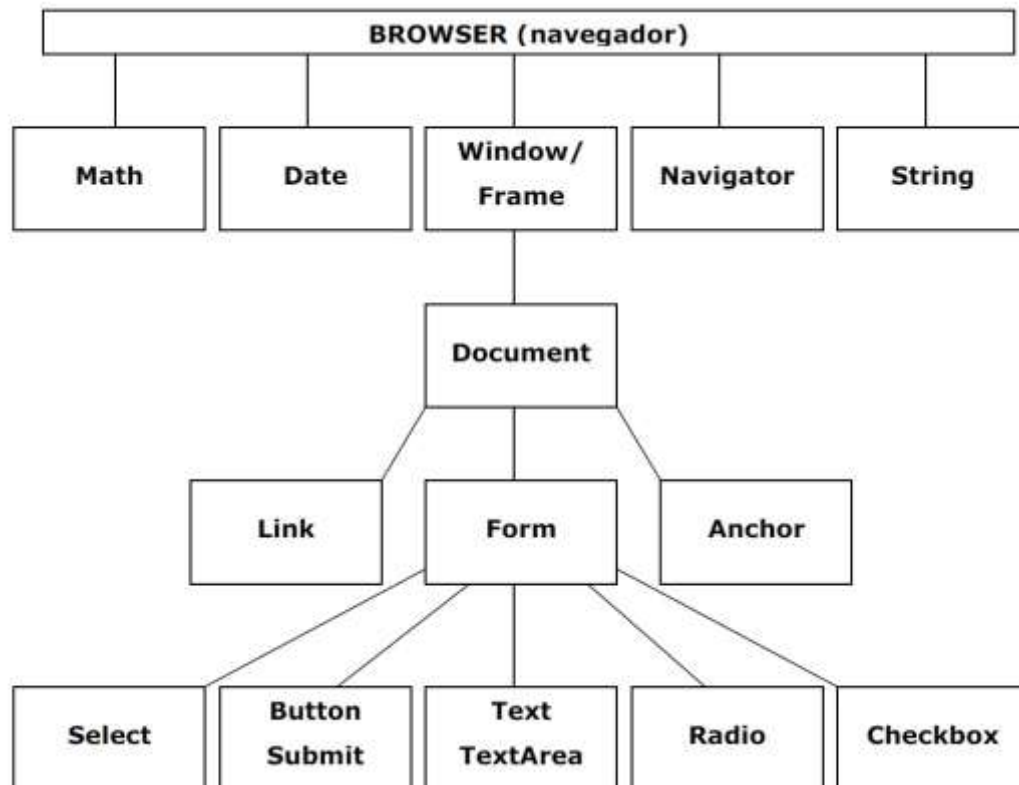
Diferente da linguagem HTML, a linguagem JavaScript corresponde à programação orientada a objetos, isto significa que todos os elementos de uma página da web são tratados como objetos. Estes objetos são agrupados de acordo com seu tipo ou finalidade. Dentro da linguagem JavaScript são criados automaticamente objetos que permitem que o usuário possa criar novos objetos de acordo com sua conveniência. Ao ser carregada uma página da Web, é criado um determinado número de objetos JavaScript, com propriedades e valores próprios que são ajustados pelo conteúdo da própria página. Todos eles seguem uma hierarquia que reflete toda a estrutura de uma página HTML. A linguagem JavaScript pode ser utilizada para a criação de scripts tanto do lado cliente como do lado servidor. Seguindo a hierarquia de objetos da linguagem JavaScript, são criados os seguintes objetos ao ser carregada uma página:

- **window:** contém propriedades que se aplicam a toda a janela. Há também um objeto desta classe para todas as "sub-janelas" de um documento com frames.
- **location:** contém as propriedades da URL atual.
- **history:** contém as propriedades das URLs visitadas anteriormente.
- **document:** contém as propriedades do documento contido na janela, tais como o seu conteúdo, título, cores, etc.

5.4.2 - MANIPULAÇÃO DE OBJETOS

A linguagem JavaScript manipula vários tipos de objetos através do uso de suas propriedades e métodos. Estes objetos são representados por uma hierarquia, fazendo com que alguns objetos se tornem propriedades de outros, observe pelo exemplo da figura a seguir esta hierarquia formada:

Hierarquia dos Objetos do JavaScript



Conforme visto no organograma apresentado, observe que existem vários objetos e muitos deles pertencem a outros, sendo chamados então de propriedades. Veja pelo exemplo do objeto FORM que possui diversas propriedades, sendo este objeto também uma propriedade do objeto DOCUMENT.

5.4.3 - PROPRIEDADES DOS OBJETOS

Cada objeto existente na manipulação do JavaScript possui propriedades (características). Exemplo: Sabemos que um documento HTML possui título e corpo. Estas características do documento podemos chamar de propriedades. Estas propriedades existem de dois tipos, algumas são os objetos propriamente ditos, enquanto outras não. Um exemplo disto é o objeto form (formulário) que é uma propriedade do objeto document (documento), conforme mostrado no organograma apresentado anteriormente. Já a propriedade de título da página (title), é pertencente ao objeto document não havendo nenhuma propriedade sobre ela. Concluindo, podemos dizer que a propriedade form do objeto document é um objeto-filho e o objeto document é o objeto-pai. Em geral, as propriedades podem conter valores (string,

números, entre outros tipos). A utilização de propriedades se dá acompanhada de seu objeto, sendo separados por um ponto apenas. Veja abaixo a sintaxe de utilização de propriedades: `nomeObjeto.propriedade;`

5.4.4 - MÉTODOS DOS OBJETOS

Além das propriedades, os objetos podem conter métodos que são funções pré-definidas pela linguagem JavaScript que irão executar determinada operação. Por exemplo, dentro de um documento o usuário poderá utilizar o método de escrever neste documento para exibir um texto qualquer. Os métodos estarão sempre associados a algum objeto presente no documento e cada método faz parte de um objeto específico. Não tente usar métodos em objetos que não o utilizam, isto faz com que a linguagem JavaScript cause erro na execução do script. Na maioria das vezes os métodos são usados para alterar o valor de uma propriedade ou executar uma tarefa específica. Veja a sintaxe de utilização dos métodos:

`nomeObjeto.metodo(argumento);`

Na sintaxe apresentada, o `nomeObjeto` faz referência ao objeto a ser utilizado e o qual sofrerá uma ação do método, já `metodo` é o nome de identificação do método usado e entre parênteses (`argumento`) é a expressão ou valor opcional que será usado para alterar sobre o objeto.

RECORDANDO

Funções são blocos de códigos que podem ser executados fazendo uma chamada da determinada função. Todos os elementos de uma página da web são tratados como objetos e todos esses objetos podem ser acessados com o JavaScript.

Capítulo 6

Operadores

6 - OPERADORES

6.1 - INTRODUÇÃO

Operadores são símbolos utilizados para atuar sobre valores.

6.2 - OPERADORES ARITMÉTICOS

Os operadores aritméticos são usados para desenvolver operações matemáticas.

Operador	Descrição	Exemplo	Resultado
+	Adição	$x = 5$ $y = 5$ $x + y$	10
-	Subtração	$x = 8$ $y = 3$ $x - y$	5
*	Multiplicação	$x = 2$ $y = 5$ $x * y$	10
/	Divisão	$x = 20$ $y = 5$ x / y	4
%	Resto	$5 \% 2$	1

6.3 - OPERADORES DE COMPARAÇÃO

Operadores de comparação, como o próprio nome diz, permitem comparar dois ou mais valores. São usados frequentemente dentro de instruções condicionais, como em comandos if.

Operador	Descrição	Exemplo	Resultado
==	É igual a.	5 == 7	Falso (false)
===	É exatamente igual a (compara o valor e o tipo do dado).	x = 5 y = "5" x === y	Falso (false)
!=	É diferente de.	7 != 8	Verdadeiro (true)
!==	É exatamente diferente a (compara o valor e o tipo do dado).	x = 5 y = "5" x !== y	Verdadeiro (true)
>	É maior que.	5 > 10	Falso (false)
<	É menor que.	5 < 10	Verdadeiro (true)
>=	É maior ou igual a.	5 >= 10	Falso (false)
<=	É menor ou igual a.	5 <= 10	Verdadeiro (true)

6.4 - OPERADORES LÓGICOS

Os operadores lógicos aceitam os valores lógicos true e false (verdadeiro e falso) como operandos e retornam valores lógicos como resultado. Os operadores lógicos base são os seguintes:

Operador	Descrição	Exemplo	Resultado	Comentário
&&	E	x = 8 y = 5 (x < 10 && y > 1)	Verdadeiro (true)	Se X for menor que 10 e Y maior que 1...
	Ou	x = 8 y = 5 (x == 7 && y == 5)	Verdadeiro (true)	Se X for igual a 7 e Y igual a 5...
!	Negação	x = 8 y = 5 !(x == y)	Verdadeiro (true)	Se X não for igual a Y...

6.5 - OPERADORES DE ATRIBUIÇÃO

Uma das coisas que os operadores podem fazer é atribuir um valor ao que estiver à sua esquerda. Se o que estiver à esquerda for uma variável, então o valor dela passará a ser aquilo que o operador forneceu; se for outro operador, o valor atribuído será usado como operando.

Operador	Exemplo	Resultado
=	$x = y$	$x = y$
+=	$x += y$	$x = x + y$
-=	$x -= y$	$x = x - y$
*=	$x *= y$	$x = x * y$
/=	$x /= y$	$x = x / y$
%=	$x \% = y$	$x = x \% y$

6.6 - OPERADORES DE INCREMENTO E DECREMENTO

Muitas vezes é preciso fazer um contador qualquer, onde se aumenta ou se diminui determinado valor até que se chegue a outro valor, explícito ou não. Nesses casos, em que se incrementa ou decrementa o valor de uma variável, é que podem ser usados os operadores de incremento ou decremento.

Entender esses operadores é fácil. Quando se incrementa o valor de uma variável, o incremento é de uma unidade; quando se decrementa, também é de uma unidade.

Operador	Exemplo	Resultado
++	$x = 5$ $x++$	6
--	$x = 5$ $x--$	4

6.7 - EXERCÍCIO PROPOSTO

- 1) Crie variáveis e utilize todos os operadores JavaScript com as variáveis criadas.
Exiba o resultado das variáveis utilizando o Alert.

Capítulo 7

Estruturas de controle

7 - ESTRUTURAS DE CONTROLE

7.1 - INTRODUÇÃO

Assim como em outras linguagens, estruturas de controle são utilizadas para executar diferentes ações baseadas em determinadas condições.

Durante o desenvolvimento dos scripts pode ser necessário executar determinadas ações baseadas em uma determinada condição, ou seja, se uma condição for verdadeira (true) será executada a ação X; caso contrário, se for falsa (false), a ação Y.

7.2 - IF

Utiliza-se tal declaração quando for necessário que somente uma parte do código seja executada e quando uma determinada condição for verdadeira.

Sintaxe:

```
if(condicao) {  
  
    //Comandos a serem executados  
  
}
```

Exemplo:

```
<script type="text/javascript">  
  
    var estado = "Sergipe";  
  
    if(estado == "Sergipe") {  
  
        alert("Sergipe");  
  
    }  
  
</script>
```

É verificado se o valor da variável estado é igual a "Sergipe", se for, será executado o Alert.

Exemplo:

```
<script type="text/javascript">

    var estado = "Sergipe";

    var pais = "Brasil";

    if(estado == "Sergipe") {

        alert("Sergipe - "+pais);

    }

</script>
```

É verificado se o valor da variável estado é igual a "Sergipe" e executado o Alert com a seguinte mensagem: "Sergipe – " + [valor da variável strpais], o resultado obtido é: "Sergipe - Brasil".

Exemplo em execução:



7.3 - IF...ELSE

Utiliza-se essa declaração para que somente uma parte do código seja executada, quando determinada condição for verdadeira; e a outra parte, quando a condição for falsa.

Sintaxe:

```
If(condição) {

    /*Comandos a serem executados quando a condição for verdadeira*/

} else {
```

```
/*Comandos a serem executados quando a condição for falsa*/
```

```
}
```

Exemplo:

```
<script type="text/javascript">

    var estado = "São Paulo";

    if(estado == "Sergipe") {

        alert("Moro em Sergipe");

    } else {

        alert("Não moro em Sergipe");

    }

</script>
```

É verificado se o valor da variável estado é igual a "Sergipe", se for, um alerta dizendo "Moro em Sergipe" será exibido; senão, será exibido o seguinte alerta: "Não moro em Sergipe".

Exemplo:

```
<script type="text/javascript">

    var idade = 18;

    if(idade >= 18) {

        alert("Jovem");

    } else {

        alert("Adolescente");

    }

</script>
```

No exemplo foi declarada a variável chamada idade, que recebeu o valor 18. Foi verificado: Se idade (≥ 18) for maior ou igual a 18, então será exibido o alerta "Jovem", senão exibirá o alerta "Adolescente".

7.4 - IF...ELSE IF...ELSE

Utiliza-se a declaração para selecionar um bloco de código entre vários outros.

Sintaxe:

```
if(condição) {

    /* Comandos a serem executados quando a condição for verdadeira*/

} else if(condição 2) {

    /*Comando a ser executado quando a segunda condição for verdadeira*/

} else {

    /*este comando será executado se a primeira e a segunda condição for falsa*/

}
```

Exemplo:

```
<script type="text/javascript">

    var estado = "Sergipe";

    if(estado == "Sergipe") {

        alert("Moro em Sergipe");

    } else if(estado == "São Paulo") {

        alert("Moro em São Paulo");

    } else {

        alert("Moro em outro estado");

    }
```

```
}
```

```
</script>
```

Exemplo:

```
<script type="text/javascript">
```

```
    var idade = 15;
```

```
    if(idade >= 18) {
```

```
        alert("Jovem");
```

```
    } else if(idade >=12) {
```

```
        alert("Adolescente");
```

```
    } else {
```

```
        alert("Criança");
```

```
}
```

```
</script>
```

Mesma linha de raciocínio do exemplo anterior.

Primeiramente é verificado se a idade é maior ou igual a 18, se for, o alerta será "Jovem", senão, será verificado se a idade é maior ou igual a 12. Se for, o alerta será "Adolescente", mas senão satisfazer a nenhuma dessas duas condições o alerta será "Criança".

7.5 - SWITCH

Utiliza-se essa declaração também para selecionar um bloco de código entre vários outros. Esta é a forma mais prática em alguns casos.

A instrução switch é similar a uma série de instruções IF seguida.

Sintaxe:

```
switch (expressão) {

    case valor1:

        //Comando a ser executado se a expressão = valor1

        break;

    case valor2:

        //Comando a ser executado se a expressão = valor2

        break;

    default:

        /*Comando a ser executado se a expressão for diferente do valor1 e valor2*/

}
```

Exemplo:

```
<script type="text/javascript">

    var estado = "SE";

    switch(estado) {

        case "SP":

            alert("São Paulo");

            break;

        case "SE":

            alert("Sergipe");

            break;

        case "RJ":

            alert("Rio de Janeiro");
```

```

        break;

        default:

            alert("Estado não encontrado");

    }

</script>

```

A variável estado tem o valor SE. Depois, inicia-se a declaração condicional com o switch, passando-lhe como parâmetro a expressão (variável) estado. Foi utilizado o case (caso). É verificado o seguinte: se o valor da variável for SE será exibido um determinado texto; se for SP será exibido outro, e assim por diante; se o valor da variável não coincidir com nenhum dos valores nos cases, será exibido um texto padrão (default).

7.5.1 - CONSIDERAÇÕES ACERCA DA ESTRUTURA SWITCH

- A expressão na maioria das vezes será uma variável.
- O valor da expressão é comparado com os valores de cada caso (case) da estrutura. Caso ocorra um valor semelhante, o código a ele associado será executado.
- O break previne que o próximo caso (case) seja executado automaticamente.
- O valor padrão (default) é utilizado caso nenhum dos valores dos cases coincida com o valor da expressão.

Exemplo:

```

<script type="text/javascript">

    var dia_da_semana = 1;

    switch(dia_da_semana) {

        case 1:

            alert("Domingo");

```

```
break;

case 2:

    alert("Segunda-feira");

break;

case 3:

    alert("Terça-feira");

break;

case 4:

    alert("Quarta-feira");

break;

case 5:

    alert("Quinta-feira");

break;

case 6:

    alert("Sexta-feira");

break;

case 7:

    alert("Sábado");

break;

default:

    alert("Dia da semana não encontrado");
```

```
}
```

```
</script>
```

7.6 - FOR

Looping que percorre determinado bloco de código por uma quantidade especificada de vezes.

O FOR é utilizado para repetir instruções em determinado número de vezes. Entre todos os loops, o FOR costuma ser utilizado quando se sabe o número exato de vezes que a sentença deve ser executada. A sintaxe do loop FOR é a seguinte:

```
for (inicição; condição, atualização) {  
  
    //Comandos a serem executados a cada repetição  
  
}
```

O FOR é constituído por três partes entre os parênteses: iniciação, condição e atualização. Iniciação se executa somente ao começar a primeira repetição do loop. Nessa parte costuma-se colocar a variável utilizada para levar a conta das vezes que o loop será executado. A segunda parte, condição, avalia se deve começar a repetição do loop. Contém uma expressão para comprovar quando deve se deter o loop, ou melhor, qual a condição que se deve cumprir para que continue a execução do loop. Por último, a atualização serve para indicar as mudanças que devem ser executadas nas variáveis, a cada vez que termine a interação do loop, antes de comprovar se deve seguir executando.

Exemplo:

Veja este exemplo de utilização do loop for, em que deverão ser impressos os números do 1 ao 100:

```
<script type="text/javascript">  
  
    for (i = 1; i<= 100; i++) {  
  
        document.write(i);  
  
        document.write("<br>");
```



```
}  
  
</script>
```

Inicia-se a variável *i* em 1. Como condição para realizar uma repetição, tem de se cumprir que a variável *i* seja menor ou igual a 100. Como atualização, incrementa-se a variável *i* em 1.

Exemplo:

```
<script type="text/javascript">  
  
    for (i=5;i<=10;i++) {  
  
        document.write("Número: " + i);  
  
        document.write("<br />");  
  
    }  
  
</script>
```

Exemplo:

Suponha que se queira, em determinada situação, os números de 1 a 500 apresentados de dois em dois. Deverá ser escrito o seguinte loop:

```
<script type="text/javascript">  
  
    for (i=1;i<=500;i+=2) {  
  
        document.write(i);  
  
        document.write("<br />");  
  
    }  
  
</script>
```

Exemplo:

Para escrever os cabeçalhos, de h1 até h6, em um documento:

```
<script type="text/javascript">
```

```

for (i=1;i<=6;i++) {

    document.write("<H" + i + "> Alfama Cursos " + "</H" + i + ">");

}

</script>

```

7.7 - WHILE

Looping que percorre determinado bloco de código se, e enquanto, a condição for verdadeira.

Deve ser utilizado para repetir, em um número indefinido de vezes, a execução de certas sentenças sempre que se cumpra determinada condição. Sim, o loop while é mais simples de compreender do que o loop FOR, pois não incorpora na mesma linha a iniciação das variáveis, a condição para executar e a atualização. Basta indicar como será demonstrada a seguir, a condição que deve ser cumprida para que se realize uma repetição:

```

while ( var <= valor_final ) {

    //Comandos a serem executados

}

```

Exemplo:

```

<script type="text/javascript">

    var i = 0;

    /* Enquanto o valor de i for menor ou igual a 10 execute os comandos abaixo */

    while (i<=10) {

        //Excreve no documento

        document.write("Número: " + i);

        document.write("<br>");

        //Alimenta o valor de i (incremento)
    }

```

```

        i++;

        //Poderia ser assim também: i = i + 1;

    }

</script>

```

O exemplo do loop while é semelhante ao do loop for. O resultado de ambos são os mesmos, porém eles se diferenciam por suas características. No exemplo acima, enquanto a variável i não for maior ou igual a 10, ele continuará executando os comandos (um laço de repetição infinito, senão houver um incremento, ele nunca satisfará a condição e pode até travar o computador ou o navegador).

Exemplo:

```

<script type="text/javascript">

    var i = 0;

    var nome = prompt("Qual seu nome ?");

    var vezes = prompt("Escrever seu nome quantas vezes ?", "100");

    /* Enquanto "i" for menor ou igual ao número de vezes que você digitou no prompt
    box, ele escreverá na tela: "Aluno da Alfama Cursos: Nome informado" */

    while (i <= vezes) {

        document.write("Aluno da Alfama Cursos:" + nome);

        document.write("<br />");

        i = i + 1;

    }

</script>

```

7.8 - DO...WHILE

Esse looping é uma variável do loop while. Ele sempre executará um bloco de código uma vez e o repetirá enquanto a condição for verdadeira. Mesmo que uma condição seja falsa ele ocorrerá uma vez, pois a condição é verificada após a execução.

Usa-se o loop do...while quando se quer que um trecho de código seja repetido várias vezes, mas que execute o bloco de código antes da verificação da condição.

Sintaxe:

```
do {  
    //Comandos a serem executados  
} while (condição)
```

Exemplo:

```
<script type="text/javascript">  
    var numero = 1;  
    do {  
        document.write("Número: "+numero);  
        numero = numero + 1;  
    } while (numero < 1);  
</script>
```

A condição (numero < 1) indica que a variável número é menor que 1, ou seja, ela é 0. Logo, o loop não deveria ser executado, mas a estrutura do...while executa os comandos primeiramente para depois verificar a condição.

7.9 - BREAK

O break interrompe o looping e prossegue com o restante do código (se houver). Instrução que aumenta o controle sobre os loops e serve para interromper repetição.

Objetivo: Deter a execução de um loop e sair dele.

Exemplo:

```
<script type="text/javascript">
    for (i=0;i<1000;i++) {
        document.write("Número: " + i + "<br>");
        var parar = prompt("Parar de escrever?", "sim");
        if (parar == "sim") {
            document.write("<br /><b />Parei de escrever no documento.");
            break;
        }
    }
</script>
```

O exemplo acima contém um loop for, que permite escrever até 1000 vezes na página. Porém, para cada vez ele perguntará se quer parar de escrever. A instrução if é utilizada para verificar se deseja continuar escrevendo no documento. Se if for sim, então BREAK (pare).

Exemplo:

```
<script type="text/javascript">
    for (numero = 995; numero < 1200; numero++) {
        if(numero == 1000) {
            break;
        }
        alert("Número: " + numero);
    }
</script>
```

Nesse exemplo, o loop for deveria ocorrer até o número 1199 (numero < 1200). Porém, utiliza-se a condição if para detectar quando a variável número estiver no número 1000 e para o looping.

7.10 - CONTINUE

É uma instrução que aumenta o controle sobre os loops, serve para continuar, reiniciar a repetição.

Objetivo: Deter a repetição atual e voltar ao início do loop.

O comando continue interrompe o loop, em determinado momento, e depois continua normalmente. Exemplo:

```
<script type="text/javascript">

    for (i=0;i<=10;i++) {

        if(i==5) {

            continue;

        }

        document.write("Alfama Cursos <b>" + i + "</b>");

        document.write("<br />");

    }

</script>
```

No exemplo acima, o loop for deve escrever no documento, do número 0 ao número 10, porém o comando continue interrompe a escrita do número 5.

Exemplo em execução:

Observe que o número 5 não foi escrito.

Alfama Cursos 0
Alfama Cursos 1
Alfama Cursos 2
Alfama Cursos 3
Alfama Cursos 4
Alfama Cursos 6
Alfama Cursos 7
Alfama Cursos 8
Alfama Cursos 9
Alfama Cursos 10

7.11 - EXERCÍCIOS PROPOSTOS

- 1) Informe um número utilizando o Prompt. Com o IF e o ELSE verifique se esse número é maior que 10 e retorne o resultado com um Alert.

- 2) Crie um Array e percorra-o utilizando o FOR. Com o Alert, informe ao usuário o valor guardado em cada índice do Array.

Capítulo 8

Objetos JavaScript (Parte I)

8 - OBJETOS JAVASCRIPT (PARTE I)

8.1 - STRING

O JavaScript é bastante poderoso no manuseio de string's (textos, caracteres), fornecendo ao programador total flexibilidade.

8.1.1 - LENGTH

Retorna o tamanho da string (números de caracteres).

```
<script type="text/javascript">

    var str = "Alfama Cursos - JavaScript";

    alert(str.length); //Resultado: 26

</script>
```

8.1.2 - CHARAT (POSIÇÃO)

Retorna o caractere da posição especificada (inicia em 0).

```
<script type="text/javascript">

    var str = "Alfama Cursos - JavaScript";

    alert(str.charAt(7)); //Resultado: C

</script>
```

8.1.3 - INDEXOF("STRING")

Retorna o número da posição onde começa a primeira "string".

```
<script type="text/javascript">

    var str = "Alfama Cursos - JavaScript";

    alert(str.indexOf("JavaScript")); //Resultado: 16
```

```
</script>
```

8.1.4 - SUBSTRING (INDEX1, INDEX2)

Retorna o conteúdo da string que corresponde ao intervalo especificado. Começa no caractere posicionado em index1 e termina em index2.

```
<script type="text/javascript">
```

```
var str = "Alfama Cursos - JavaScript";
```

```
alert(str.substring(0,5)); //Resultado: Alfama
```

```
</script>
```

8.1.5 - TOUPPERCASE()

Passa o conteúdo da string para letra maiúscula (Caixa Alta).

```
<script type="text/javascript">
```

```
var str = "Alfama Cursos - JavaScript";
```

```
alert(str.toUpperCase()); //Resultado: ALFAMA CURSOS - JAVASCRIPT
```

```
</script>
```

8.1.6 - TOLOWERCASE()

Passa o conteúdo da string para letra minúscula (Caixa Baixa).

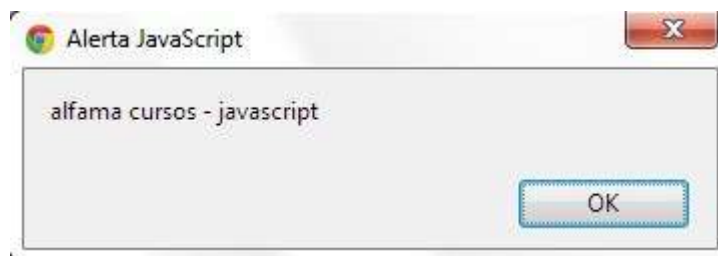
```
<script type="text/javascript">
```

```
var str = "alfama cursos - javascript";
```

```
alert(str.toLowerCase());
```

```
</script>
```

Em execução:



8.1.7 - ESCAPE()

Retorna o valor ASCII da string (vem precedido de %).

```
<script type="text/javascript">

    var str = "Alfama Cursos - JavaScript";

    alert(escape(str));

</script>
```

8.1.8 - REPLACE()

Sintaxe:

```
str.replace ("procurar por", "substituir por")
```

str neste caso é uma menção à variável chamada str, e não se trata de nenhuma palavra reservada ou objeto próprio da linguagem JavaScript, ou seja, se você tivesse uma variável chamada "nome" e quisesse usar a função replace nela, faria assim: nome.replace("procurar por","substituir por"). Substitui alguns caracteres por outros em uma string.

```
<script type="text/javascript">

    var str = "Alfama Cursos - JavaScript";

    alert(str.replace("JavaScript","PHP")); //Resultado: Alfama Cursos - PHP

</script>
```

8.2 - MATH

O objeto Math é utilizado para realizar tarefas matemáticas, fornecendo diversos valores e funções.

8.2.1 - MATH.ABS(NÚMERO)

Retorna o valor absoluto do número (ponto flutuante).

8.2.2 - MATH.CEIL(NÚMERO)

Arredonda o valor informado para cima, até o número inteiro mais próximo.

```
<script type="text/javascript">

    var valor = 7.3;

    alert(Math.ceil(valor)); //Resultado: 8

</script>
```

8.2.3 - MATH.FLOOR(NÚMERO)

Arredonda o valor informado para baixo até o número inteiro mais próximo.

```
<script type="text/javascript">

    var valor = 7.3;

    alert(Math.floor(valor)); //Resultado: 7

</script>
```

8.2.4 - MATH.ROUND(NÚMERO)

Retorna o valor inteiro, arredondado, do número.

8.2.5 - MATH.POW(BASE, EXPOENTE)

Retorna o cálculo do exponencial.

8.2.6 - MATH.MAX(NÚMERO1, NÚMERO2)

Retorna o maior número entre os dois fornecidos.

```
<script type="text/javascript">

    var valor1 = 7.8;

    var valor2 = 7.3;

    alert(Math.max(valor1,valor2)); //Resultado: 7.8

</script>
```

8.2.7 - MATH.MIN(NÚMERO1, NÚMERO2)

Retorna o menor número entre os dois fornecidos.

```
<script type="text/javascript">

    var valor1 = 7.8;

    var valor2 = 7.3;

    alert(Math.min(valor1,valor2)); //Resultado: 7.3

</script>
```

8.2.8 - MATH.SQRT(NÚMERO)

Retorna a raiz quadrada do número.

8.2.9 - MATH.SIN(NÚMERO)

Retorna o seno de um número (ângulo em radianos).

8.2.10 - MATH.ASIN(NÚMERO)

Retorna o arco seno de um número (em radianos).

8.2.11 - MATH.COS(NÚMERO)

Retorna o cosseno de um número (ângulo em radianos).

8.2.12 - MATH.ACOS(NÚMERO)

Retorna o arco cosseno de um número (em radianos).

8.2.13 - MATH.TAN(NÚMERO)

Retorna a tangente de um número (ângulo em radianos).

8.2.14 - MATH.ATAN(NÚMERO)

Retorna o arco tangente de um número (em radianos).

8.2.15 - MATH.PI

Retorna o valor de PI (aproximadamente 3,14159).

8.2.16 - MATH.LOG(NÚMERO)

Retorna o logaritmo de um número.

8.2.17 - MATH.E

Retorna a base dos logaritmos naturais (aproximadamente 2.718).

8.2.18 - MATH.RANDOM()

Retorna um número entre 0 e 1.

```
<script type="text/javascript">
```

```
    alert(Math.random()); //Exibirá números aleatórios entre 0 e 1: Ex:
    0.10937464829
```

```
    alert(Math.random()*10); //Exibirá números aleatórios entre 0 e 10: Ex:
    9.10937464829
```

```
</script>
```

8.3 - EXERCÍCIOS PROPOSTOS

- 1) Crie uma variável com uma string e utilize todos os métodos de String. Visualize o resultado utilizando o Alert.

- 2) Utilizando os métodos do objeto MATH, faça cálculos matemáticos e exiba esses valores utilizando o Alert.

Capítulo 9

Objetos JavaScript (Parte II)

9 - OBJETOS JAVASCRIPT (PARTE II)

9.1 - DATE

O objeto Date() permite trabalhar com datas e horas. Para criá-lo, utiliza-se a palavra-chave new.

```
<script type="text/javascript">
```

```
    var data = new Date();
```

```
    alert(data);
```

```
</script>
```

Existem formas para que a data seja exibida na maneira desejada. Veja como exibir a data com o seguinte formato: dd/mm/aaaa.

```
<html>
```

```
    <head>
```

```
        <title> Alfama Cursos - JavaScript </title>
```

```
    </head>
```

```
    <body>
```

```
        <script type="text/javascript">
```

```
            var data = new Date();
```

```
            var mes = new Array(12)
```

```
            mes[0] = "Janeiro";
```

```
            mes[1] = "Fevereiro";
```

```
            mes[2] = "Março";
```

```
            mes[3] = "Abril";
```

```
            mes[4] = "Maio";
```

```

mes[5] = "Junho";

mes[6] = "Julho";

mes[7] = "Agosto";

mes[8] = "setembro";

mes[9] = "Outubro";

mes[10] = "Novembro";

mes[11] = "Dezembro";

document.write("Hoje é: <b>");

    document.write(data.getDate() + " - " + mes[data.getMonth()] + " - " +
data.getFullYear());

</script>

</body>

</html>

```

No exemplo acima foi utilizado o objeto Date e três de seus métodos: **getDate()**, **getMonth()** e **getFullYear()**. O primeiro retorna o dia do mês (de 1 a 31), o segundo o mês (de 0 a 11) e o último o ano com quatro dígitos. Foi utilizado um Array para o mês, uma vez que o método **getMonth()** retorna números de 0 (zero) a 11 (onze). Nessa lógica, o mês de novembro é representado pelo número 10 (dez). Assim, utiliza-se um Array para setar o nome do mês (pode ser em algarismo também), de acordo com o número retornado pelo método.

9.1.1 - MÉTODOS DO OBJETO DATE()

O objeto Date() em si retorna a data e o horário atual. Date() é um objeto e dispõe de muitos métodos. Veja os mais utilizados:

- **getDate()**

Retorna o dia do mês (1-31).

- **getDay()**

Retorna o dia da semana (0-6).

- **getMonth()**

Retorna o mês (0-11).

- **getFullYear()**

Retorna o ano com quatro dígitos.

- **getHours()**

Retorna a hora (0-23).

- **getMinutes()**

Retorna os minutos (0-59).

- **getSeconds()**

Retorna os segundos (0-59).

- **getMilliseconds()**

Retorna os milissegundos (0-999).

- **getTimezoneOffset()**

Retorna a diferença em minutos entre o tempo local e o do Meridiano de *Greenwich* (GMT).

Exemplo de utilização dos métodos acima:

```
<script>

    var data = new Date();

    data.NomeDoMetodo();

</script>
```

Exemplo prático:

```
<html>

  <head>

    <title> Alfama Cursos - JavaScript </title>

    <script type="text/javascript">

      data = new Date();

      document.write( data.getDay());

    </script>

  </head>

  <body>

  </body>

</html>
```

9.2 - DOCUMENT

O objeto document faz parte do objeto window, sendo obtido como uma propriedade deste, na forma window.document. Este objeto contém uma representação de um documento escrito em HTML.

Se escrevermos apenas document, omitindo o objeto window, obteremos o objeto que representa o documento da página que estamos usando. Se especificarmos um objeto window correspondente à outra janela ou à outra moldura, então iremos obter o objeto document correspondente à página que está nessa janela ou moldura.

9.2.1 - PROPRIEDADES DO OBJETO DOCUMENT

Propriedade	Descrição
body	Devolve o elemento <body> (se o documento estiver escrito em HTML sem molduras) ou o elemento <frameset> (se o documento estiver escrito em HTML Frameset).
cookie	Devolve os <i>cookies</i> associados ao documento.
domain	Lê o nome de domínio do servidor que enviou o documento.
referrer	Lê o endereço (URL) da página em que está a ligação que o conduziu ao documento atual.
title	Lê ou define o título do documento.
URL	Lê o endereço (URL) do documento.

9.2.2 - COLEÇÕES DO OBJETO DOCUMENT

Coleção	Descrição
anchors	Contém uma lista com todos os elementos <a>(âncoras) existentes no documento.
applets	Contém uma lista com todos os elementos <applet> (miniaplicações escritas em Java).
embeds	Contém uma lista com todos os objetos que foram embutidos na página usando os elementos <embed> ou <object>.

forms	Contém uma lista com todos os elementos <form> existentes no documento.
images	Contém uma lista com todas as imagens existentes no documento.
links	Contém uma lista com todos os elementos <a> existentes no documento que especificam um valor para o atributo href (é uma subcoleção de anchors).
plugins	Contém a mesma lista que a coleção embeds.

9.2.3 - MÉTODOS DO OBJETO DOCUMENT

Métodos	Descrição
close()	Fecha um documento que tenha sido aberto com o método document.open() e escrito com document.write(). Não confundir com o método open() do objeto window, que serve para abrir uma nova janela.
getElementById()	Devolve o elemento que possui o valor especificado no atributo id.
getElementsByName()	Contém uma lista com todos os elementos que têm um determinado nome (dado por uma etiqueta do HTML).
open()	Abre um documento para nele escrever conteúdo novo.
write()	Escreve texto num documento.
writeln()	Escreve uma linha de texto num documento.

Exemplo:

Ler o título do documento.

```
<html>
```

```

<head>

    <title> Alfama Cursos - JavaScript </title>

</head>

<body>

    <p>

        O título desta página é:

        <script type="text/javascript">

            document.write(document.title)

        </script>

    </p>

</body>

</html>

```

9.3 - EXERCÍCIO PROPOSTO

- 1) Retorne a data atual no formato dd/mm/aaaa, utilizando o objeto DATE e escreva a data retornada na tela utilizando o objeto DOCUMENT.

Capítulo 10

Objetos JavaScript (Parte III)

10 - OBJETOS JAVASCRIPT (PARTE III)

10.1 - LOCATION

O objeto location faz parte do objeto window, sendo obtido como uma propriedade deste, na forma window.location. Este objeto contém o URL completo da página que está carregada numa determinada janela.

Se escrevermos apenas location, omitindo o objeto window, obteremos o objeto correspondente à página que estamos usando. Se especificarmos um objeto window, então iremos obter o objeto location correspondente à página que está na janela que indicamos.

Todas as propriedades deste objeto possuem valores formados por texto.

10.1.1 - PROPRIEDADES DO OBJETO LOCATION

Métodos	Descrição
hash	A propriedade hash é composta pela parte do endereço da página (URL) que começa pelo símbolo #.
host	Esta propriedade contém apenas a parte inicial do endereço. É composta pelo protocolo, pelo nome do servidor e eventualmente pelo número da porta usada para comunicar com o servidor.
hostname	Esta propriedade contém apenas o nome do servidor.
href	Contém o endereço completo da página. Podemos dar como valor o endereço de uma nova página para que ela seja carregada em substituição da atual. As restantes propriedades são extraídas do valor desta propriedade.
pathname	A porção do URL que vem após o valor da propriedade host. Esta propriedade contém o local em que a página se encontra dentro do servidor a que pertence.

port	Número da porta usada para comunicar com o servidor.
protocol	Parte inicial do URL (normalmente http://).
search	Parte do URL que começa com o símbolo ? (ponto de interrogação) termina no URL ou logo que seja encontrado o símbolo # (se ele estiver presente).

10.1.2 - MÉTODOS DO OBJETO LOCATION

Métodos	Descrição
reload()	Refresca a página fazendo um novo pedido de envio ao servidor.
replace()	Substitui a página atual por uma outra sem adicionar o endereço da página atual à história da navegação.

Exemplo:

Propriedades do objeto location para a página:

```
<html>

  <head>

    <title> Alfama Cursos - JavaScript </title>

  </head>

  <body>

    <script type="text/javascript">

      var s="<b>host</b> "+location.host+"<br>";

      s += "<b>hostname</b> "+location.hostname+"<br>";

      s += "<b>href</b> "+location.href+"<br>";

      s += "<b>hash</b> "+location.hash+"<br>";

      s += "<b>pathname</b> "+location.pathname+"<br>";
```

```
s += "<b>protocol</b> "+location.protocol+"<br>";
s += "<b>search</b> "+location.search+"<br>";
s += "<b>port</b> "+location.port;
document.write(s);
</script>
</body>
</html>
```

10.2 - WINDOW

O window é o objeto de topo no DOM do DHTML. Ele contém informação acerca da janela do browser e do seu conteúdo: tamanho, posição, documento, molduras (frames).

As tabelas seguintes mostram as propriedades e os métodos mais importantes deste objeto. Mais abaixo você encontrará diversos exemplos que ilustram a forma como alguns destes métodos e propriedades são usados na prática.

10.2.1 - PROPRIEDADES DO OBJETO WINDOW

Métodos	Descrição
closed	Indica (true ou false) se a janela está fechada ou não.
defaultStatus	Lê ou define a mensagem que por omissão aparece escrita na barra de status do browser.
document	Devolve o objeto document, que representa o documento que está sendo apresentado na janela.
event	Devolve o objeto event, que descreve o último acontecimento que ocorreu na janela.
history	Objeto que contém uma lista com os endereços (URLs) das páginas já visitadas na presente sessão e permite regressar a elas.
length	Lê a quantidade de molduras (frames) que estão na janela.

location	Contém informação acerca do endereço (URL) da página que está sendo apresentada e permite alterá-lo.
name	Lê ou define o nome da janela.
navigator	Devolve o objeto navigator.
opener	Devolve (se existir) o objeto que abriu a janela (só em janelas popup).
outerHeight	(Só Mozilla/Netscape e Opera) Contém a altura exterior da janela do browser.
outerWidth	(Só Mozilla/Netscape e Opera) Contém a largura exterior da janela do browser.
pageXOffset	(Só Mozilla/Netscape e Opera) Contém a coordenada x do canto superior esquerdo da janela.
pageYOffset	(Só Mozilla/Netscape e Opera) Contém a coordenada y do canto superior esquerdo da janela.
parent	Caso a página esteja numa moldura (frame ou iframe) devolve o objeto window que a contém.
returnValue	Lê o valor devolvido por uma caixa de diálogo.
screen	Devolve um objeto que contém informação acerca da tela utilizada para ver a página.
screenLeft	(Só MSIE) Contém a coordenada x do canto superior esquerdo da área da janela que contém a página (difere de pageXOffset).
screenTop	(Só MSIE) Contém a coordenada y do canto superior esquerdo da área da janela que contém a página (difere de pageYOffset).

self	Devolve o objeto window em que a página se encontra.
status	Lê ou define a mensagem que aparece escrita na barra de estado.
top	Devolve o objeto window que ocupa a posição mais elevada na hierarquia de molduras.

10.2.2 - COLEÇÕES DO OBJETO WINDOW

Métodos	Descrição
frames	Contém uma lista com todos os objetos que representam molduras que foram criadas usando os elementos <frame> ou <iframe>. Os objetos desta lista são os próprios objetos window (a cada moldura corresponde uma janela).

10.2.3 - MÉTODOS DO OBJETO WINDOW

Métodos	Descrição
alert()	Mostra uma caixa com uma mensagem de texto e um botão OK.
blur()	A janela perde o foco.
clearInterval()	Anula uma ação do método setInterval().
clearTimeout()	Anula uma ação do método setTimeout().

close()	Fecha a janela.
confirm()	Mostra uma caixa com uma mensagem de texto, um botão para Cancelar e um botão para aceitar (OK).
escape()	Codifica uma string (texto) para que ela possa ser enviada como parte do URL ou guardada num <i>cookie</i> .
focus()	Dá o foco a uma janela.
moveBy()	Desloca a janela relativamente à sua posição atual.
moveTo()	Movimenta a janela para a posição especificada.
open()	Abre uma nova janela (popup).
print()	Imprime o documento que se encontra na janela.
prompt()	Mostra uma caixa com uma pergunta e um campo para inserir uma resposta.
resizeBy()	Altera o tamanho da janela somando aos valores atuais os valores x e y especificados.
resizeTo()	Altera o tamanho da janela para os valores x e y especificados.
scrollBy()	Desloca o início da parte visível da janela somando os valores x, y às coordenadas da parte que está atualmente no início da parte visível.
scrollTo()	Faz com que a parte visível do conteúdo da janela se inicie na parte da página que tem as coordenadas (x, y) especificadas.
setInterval()	Executa a função indicada a intervalos regulares (medidos em milissegundos)

setTimeout()	Executa a função indicada uma só vez depois de passado algum tempo (medido em milisegundos).
unescape()	Descodifica uma string (texto) que estava preparada para ser enviada como parte do URL, colocando-a na forma normal. Efetua a operação inversa do método escape().

Exemplo:

Imprimir uma página:

```
<html>

  <head>

    <script type="text/javascript">

      function printPage() {

        window.print()

      }

    </script>

    <title> Alfama Cursos - JavaScript </title>

  </head>

  <body>

    <p>Clique no botão para imprimir esta belíssima página.</p>

    <input onclick="printPage()" type="button" value="Imprimir esta página">

  </body>

</html>
```

Capítulo 11

Eventos JavaScript (Parte I)

11 - EVENTOS JAVASCRIPT (PARTE I)

11.1 - INTRODUÇÃO

Evento é uma ação. Pode ser um clique com o mouse, uma tecla pressionada, o carregamento da página ou seu redimensionamento da página, passar o cursor sobre algo, enfim, muitas coisas podem ser tratadas como eventos.

11.2 - ONCLICK

O evento onClick é utilizado em links, botões, rádio, checkbox, reset. Ocorre quando o mouse clica em algum objeto.

Exemplo:

```
<html>

    <head>

        <title> Alfama Cursos - JavaScript </title>

    </head>

    <body onload="AoEntrar();">

        <input type="button" name="botao" value="Clique Aqui" onClick="alert(Alfama
Cursos - JavaScript ')"></body>

</html>
```

11.3 - ONKEYDOWN, ONKEYPRESS E ONKEYUP

Esses eventos são utilizados em associação com o teclado. Com eles é possível, por exemplo: contar os caracteres de uma textarea/edit, realizar o preview de algum texto, efetuar uma busca instantânea, dentre outras opções.

Os eventos **onkeydown** e **onkeypress** são semelhantes e ocorrem quando uma tecla é pressionada pelo usuário.

Já o **onkeyup** é executado quando a tecla for pressionada e, em seguida, liberada.

Exemplo:

```
<html>

  <head>

    <title> Alfama Cursos - JavaScript </title>

  </head>

  <body>

    <script type="text/javascript">

      function AoDigitar() {

        alert("Você apertou uma tecla");

      }

    </script>

    <strong>Digite:</strong>

    <br>

    <input type="text" name="txtedit" size="20" onKeyDown="AoDigitar();">

  </body>

</html>
```

Ao digitar no input, a função é chamada e exibirá um alerta informando que você digitou.

11.4 - ONMOUSEOVER E ONMOUSEOUT

O evento onmouseover é executado quando o mouse se localiza na área de um elemento, já o onmouseout é executado quando o mouse sai dessa área.

Exemplo:

```
<html>

  <head>

    <title> Alfama Cursos - JavaScript </title>

  </head>

  <body>
```

```

<script type="text/javascript">

    function MouseEmCima() {

        alert("Você passou o cursor em cima do link.");

    }

    function MouseFora() {

        alert("Você tirou o cursor de cima do link.");

    }

</script>

<a href="http://www.alfamacursostecnicos.com.br"
onmouseover="MouseEmCima()" onmouseout="MouseFora()">Alfama Cursos</a>

</body>

</html>

```

Ao passar o cursor do mouse em cima de um link aciona-se o evento **onmouseover**, que faz a chamada à função, exibindo um alerta dizendo que o cursor está em cima do link.

Assim que se retirar o mouse de cima do link aciona-se o evento **onmouseout**, que faz a chamada à função, exibindo um alerta dizendo que o cursor do mouse não está mais em cima do link.

Capítulo 12

Eventos JavaScript (Parte II)

12 - EVENTOS JAVASCRIPT (PARTE II)

12.1 - ONSUBMIT

Para realizar a validação de um formulário utiliza-se o evento onsubmit, o que significa dizer: ao enviar os dados do formulário, ou seja, esse evento é executado "Ao submeter o formulário ..."

Exemplo:

Para este exemplo será usado um arquivo de script (.JS), que deve ser importado para a página (conforme explicado no início deste curso). Este é um ótimo método quando se tem grande quantidade de scripts (funções), etc.

validar.js

```
function ChecarFormulario() {

    //Senão preencheu o campo nome, então ...

    if (document.formdados.txtnome.value == "") {

        //alerta ...

        alert("Informe seu nome!");

        //mude o foco para o txtnome

        document.formdados.txtnome.focus();

        return false;

    }

    //Senão preencheu o campo email, então ...

    if(document.formdados.txtemail.value == "") {

        //alerta ...

        alert("Informe seu Email !");
```

```
//mude o foco para o txtemail

document.formdados.txtemail.focus();

return false;

}

//Senão alerta que está tudo correto !

alert("Tudo correto !!!");

}

index.html

<html>

<head>

<title> Alfama Cursos - JavaScript </title>

</head>

<body>

<script type="text/javascript" src="validar.js"> </script>

<!-- Formulário -->

<form name="formdados" onSubmit="return ChecarFormulario();">

<!-- Inputs -->

Nome: <input type="text" name="txtnome" size="20"><br>

Email: <input type="text" name="txtemail" size="20"><br><br>

<!-- Botão enviar -->

<input type="submit" value="Enviar" name="btenviar">

</form>

</body>
```

</html>

Esse evento trabalha em conjunto com uma função da seguinte forma: associa-se ao formulário a chamada de uma função que tem por objetivo validar os dados submetidos e retornar um valor verdadeiro (true) ou falso (false). Se o valor retornado for verdadeiro, o formulário será enviado; caso contrário, o envio será bloqueado até que os dados sejam preenchidos corretamente.

Explicação do exemplo:

No exemplo acima foi criada uma função para validação dos campos; se algum campo não for preenchido, ela alertará e colocará o foco sobre ele.

O evento **Onsubmit** foi usado no formulário, ou seja, "ao submeter o formulário execute esta função...", o que, neste caso, executará a função de validação.

12.2 - ONFOCUS, ONBLUR E ONCHANGE

O evento **onfocus** ocorre quando o usuário clica em algum campo de texto ou link e o foco é mantido até que outro elemento o ganhe. Já o **onblur** é executado quando o elemento perde o foco.

Um exemplo para o evento **onchange** (ao mudar/trocar) é quando se altera uma opção na lista de um combobox. Nessa ação ocorre o **onchange**.

Exemplo:

```
<html>

  <head>

    <title> Alfama Cursos - JavaScript </title>

  </head>

  <body>

    <script type="text/javascript">

      <!-- Formulário -->

      function ChangeCursos() {
```

```

        alert("Mudou ...");

    }

</script>

<!-- Formulário: formcursos -->

<form name="formcursos">

    <select size="1" name=" cursos" 20. onChange="ChangeCursos();">

        <!-- itens -->

        <options>HTML</option>

        <option selected>Javascript</option>

        <option>PHP</option>

        <option>CSS</option>

        <!-- /itens -->

    </select>

</form>

</body>

</html>

```

12.3 - ONLOAD

O evento onload é executado quando uma página HTML é carregada por completo, incluindo as imagens.

O **onload** trabalha na entrada da página.

Exemplo:

```

<html>

    <head>

```



```

        <title> Alfama Cursos - JavaScript </title>

</head>

<body onload="AoEntrar();">

        <script type="text/javascript">

                function AoEntrar() {

                        alert("Seja bem vindo!");

                }

        </script>

        <b> Alfama Cursos - JavaScript</b>

</body>

</html>

```

12.4 - ONFOCUS

Ocorre quando o usuário clica em um link ou em um campo de texto.

Exemplo:

```

<html>

        <head>

                <title> Alfama Cursos - JavaScript </title>

        </head>

        <body onload="AoEntrar();">

                <input type="text" name="nome" onFocus= "alert('Digite seu nome completo')">

        </body>

</html>

```

12.5 - EXERCÍCIOS PROPOSTOS

- 1) Crie um botão HTML com o valor “IMPRIMIR” e ao clicar nesse botão utilize o método print do objeto window para imprimir a página.

- 2) Crie um validador JavaScript pra um formulário HTML.

REFERÊNCIAS BIBLIOGRÁFICAS

FLANAGAN, David. *JavaScript: o guia definitivo*. 4 e.d. Porto Alegre: Bookman, 2004.

OLIVIERO, Carlos A. J. *JavaScript - Orientado por Projeto*. 2 e.d. São Paulo: Érica, 2001.



alfamacursos.com.br